

PES UNIVERSITY

Department of Computer Science and Engineering

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Customizable Subscription Box Scheduler

Submitted by: Vinodkumar L | Jira Project: SE LAB (SCRUM)

SRN: PES1UG25AM817

1. Epics

Three Epics were created in Jira, derived from the functional requirements identified in the Lab 1 problem statement (Customizable Subscription Box Scheduler).

Epic 1 (SCRUM-5): Subscription Customization & Billing Cycle Control

Enable subscribers to customize their monthly box, view current selections, and pause or skip billing cycles. | Priority: High

Epic 2 (SCRUM-6): Promo Code Application & Billing Processing Enable subscribers to apply promo codes at checkout, view order/billing history, and allow the Payment/Billing Service to process charges. | Priority: Medium

Epic 3 (SCRUM-7): Fulfillment Manifest & Shipping/Inventory Management

Enable the Fulfillment Lead to generate the fulfillment manifest within performance SLAs and manage shipping/inventory operations. | Priority: High

Backlog: Epics & User Stories (Jira Screenshot)

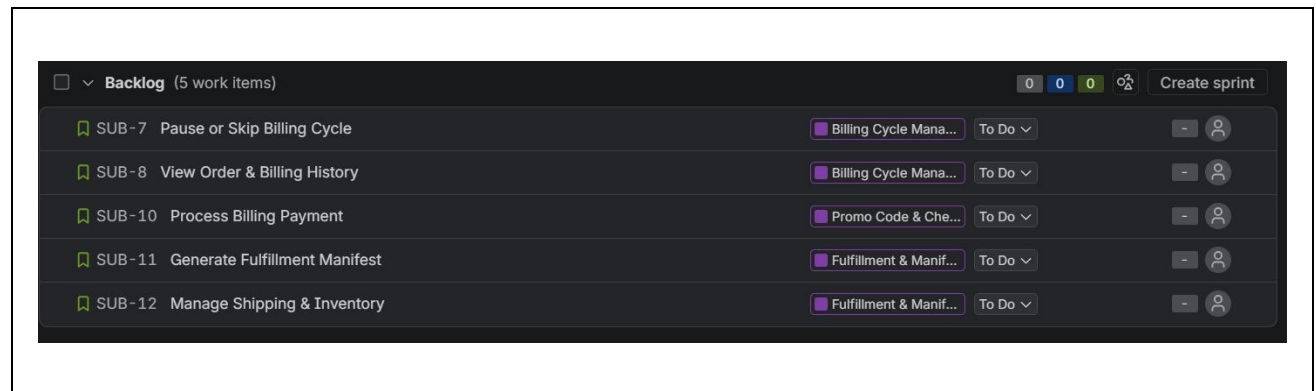


Figure 1 — Jira Backlog showing all 3 Epics with their linked User Stories, priorities, and story points.

2. User Stories & Story Point Estimation

Story points were assigned using the Fibonacci scale (3, 5, 8) based on complexity, effort, and uncertainty.

Epic 1: Subscription Customization & Billing Cycle Control

Key	Story	Priority	Points
SCRUM-8	As a subscriber, I want to customize my monthly box selection based on preference tags, so that the box matches what I actually want.	High	5
SCRUM-9	As a subscriber, I want to pause or skip an eligible billing cycle, so that I am not charged when I don't need a box.	High	5
SCRUM-10	As a subscriber, I want to view my current box selection and preference tags before customizing, so that I can make informed changes.	Medium	3

Epic 2: Promo Code Application & Billing Processing

Key	Story	Priority	Points
SCRUM-11	As a subscriber, I want to apply a valid promo code at checkout, so that I can see the discounted final charge before confirming.	Medium	5
SCRUM-12	As the Payment/Billing Service, I want to process or confirm billing transactions for renewals and promo-adjusted charges, so that payments are accurate and reliable.	High	8
SCRUM-13	As a subscriber, I want to view my order and billing history, so that I can track past charges and deliveries.	Medium	3

Epic 3: Fulfillment Manifest & Shipping/Inventory Management

Key	Story	Priority	Points
SCRUM-14	As a Fulfillment Lead, I want to generate the fulfillment manifest for the current billing cycle, so that all eligible boxes and selected items are ready for shipment.	High	8
SCRUM-15	As a Fulfillment Lead, I want the manifest generator to export shipping labels for 10,000 boxes in under a minute, so that peak-load fulfillment stays on schedule.	High	5
SCRUM-16	As a Fulfillment Lead, I want to manage shipping and inventory operations, so that stock levels and shipments stay accurate and up to date.	Medium	5

3. Sprint Simulation

SCRUM Sprint 1 was created with a 1-week duration (Sep 1 – Sep 15, 2026) and populated with backlog items including the new User Stories, prioritizing Epic 1 (Subscription Customization & Billing Cycle Control) since it is a prerequisite for the rest of the workflow.

Sprint Board — Initial State (Sprint Started)

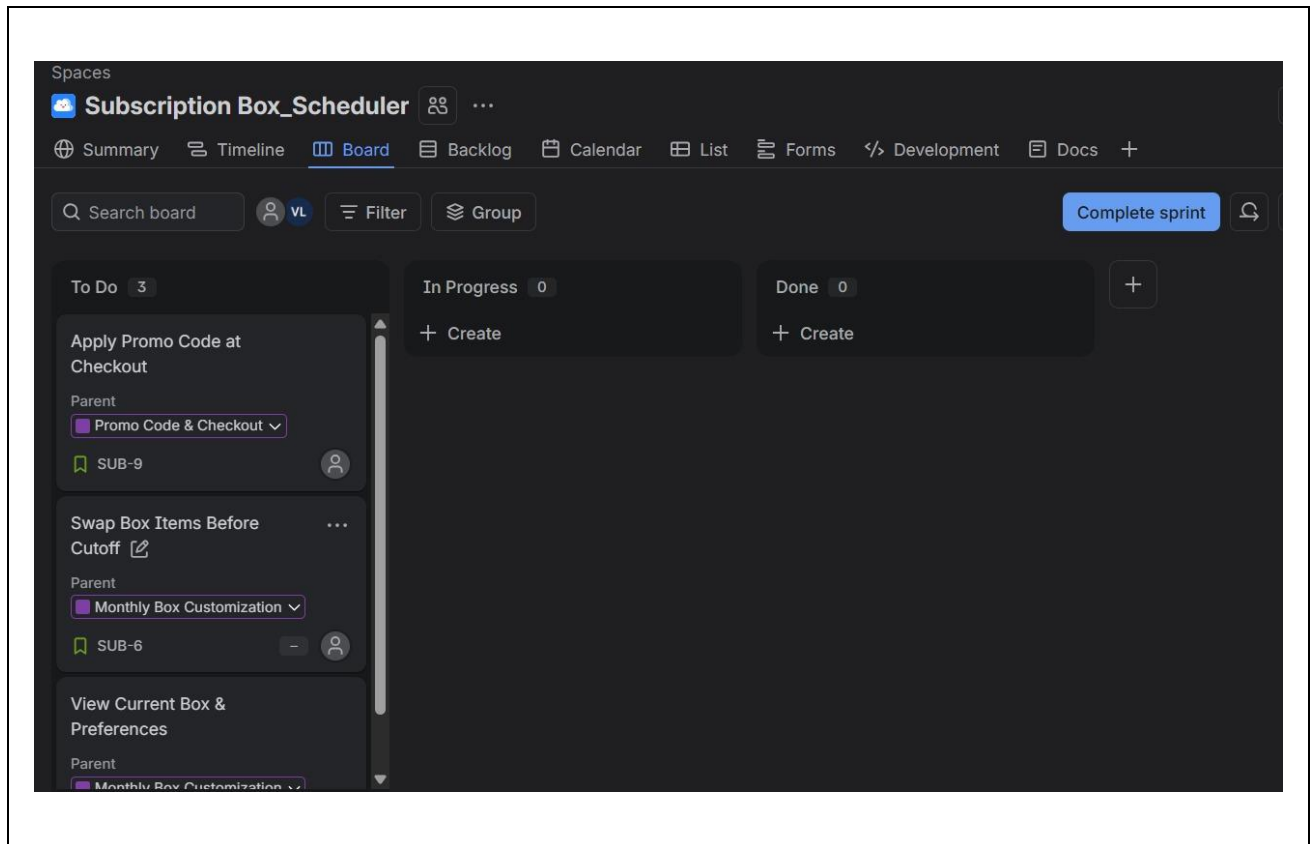


Figure 2 — Active Sprint board immediately after Start Sprint, with items in To Do and In Progress.

Sprint Board — Progressing Tasks (To Do → In Progress → Done)

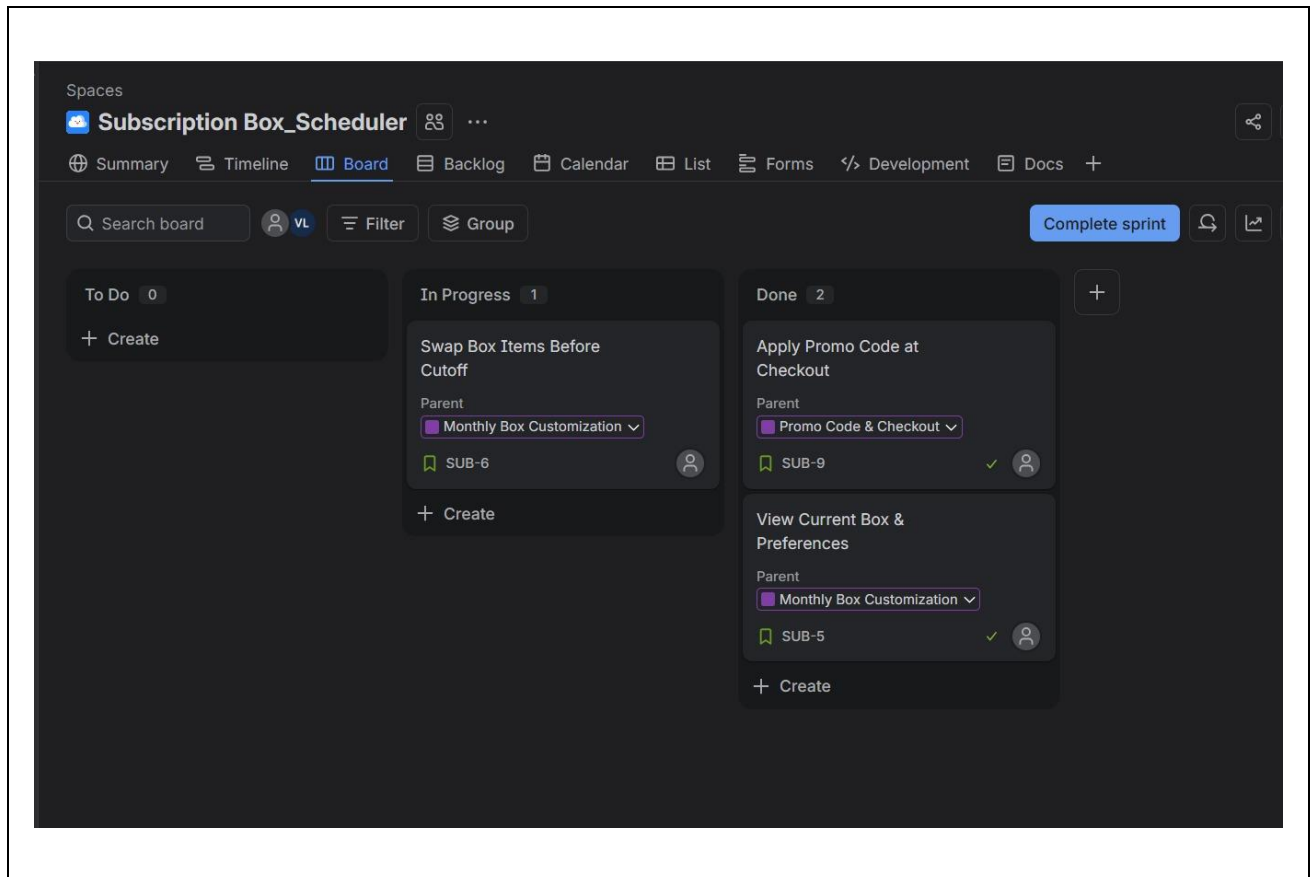


Figure 3 — Stories 1.1 and 1.2 moved into In Progress (later marked Done, reflected in the burndown chart below) while Story 1.3 remains in To Do, simulating real sprint progress.

4. Burndown Chart

The Burndown Chart (Reports → Burndown Chart) tracks remaining story points against the ideal guideline over the sprint duration.

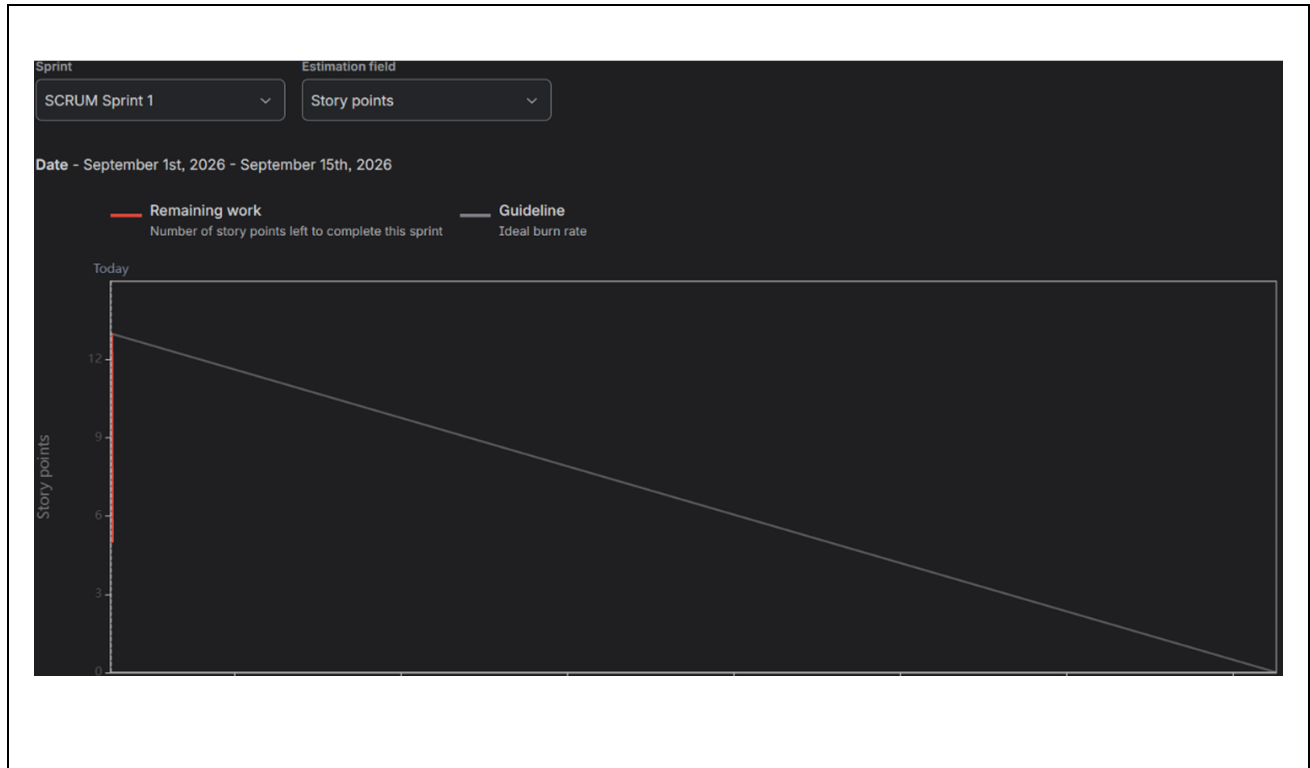


Figure 4 — Sprint burndown showing remaining work dropping from 13 points as Stories 1.1 (5 pts) and 1.2 (5 pts) were completed, against the ideal guideline line.

5. Reflection Questions

1. Did your estimations reflect the actual effort?

The Fibonacci-based estimates (3, 5, 8) captured relative complexity reasonably well. Simpler display-only stories, such as viewing the current box selection (3 pts), were estimated lower than stories with more moving parts, like billing transaction processing (8 pts) or fulfillment manifest generation under a strict performance SLA (8 pts), which involve validation logic, edge cases, and timing constraints. In a real team setting, planning poker with multiple estimators would likely refine these numbers further, especially for the higher-uncertainty billing and fulfillment items.

2. Was your backlog well-prioritized?

Yes — subscription customization and billing-cycle control (Epic 1) were marked High priority since they are the core interaction subscribers rely on every cycle. Fulfillment manifest generation (Epic 3) was also High priority since fulfillment cannot begin until selections are finalized and the manifest is exported within the performance requirement. Promo code application and billing history (Epic 2) were set to Medium, as they are enhancements to the checkout flow rather than blockers to the core subscription workflow. This reflects a logical dependency order: subscribers must finalize their box before billing is processed, and fulfillment depends on both being complete.

3. How did your simulated sprint align with your plan?

Sprint 1 planned to complete the three Subscription Customization & Billing Cycle Control stories (13 points total). Two of the three (Stories 1.1 and 1.2, 10 points combined) were moved to Done during the simulation, while Story 1.3 remained in To Do. This is a realistic outcome — teams often carry over one or two items to the next sprint rather than completing 100% of planned scope, especially when a story like viewing preference tags depends on the customization UI being finalized first.

4. What insights did the burndown chart give about your team's capacity?

The chart's remaining-work line dropped below the ideal guideline early, as the two customization and pause/skip stories were completed close together, then flattened — indicating the last story wasn't finished within the visible window. This pattern suggests capacity was front-loaded: the higher-priority customization and pause/skip stories were completed first, while the smaller preference-tag display story was left for later despite its lower point value. It signals that future sprints should sequence work by dependency rather than just priority, so effort is spread more evenly and no story is left idle purely because it was ordered last.